

Module 9 Week A — Core Skills Drill: SPARQL on a Fixture KG

Due: Tonight, after the concept session. Resubmissions are accepted through Saturday of the assignment week.

The drill takes the SPARQL grammar you read about (`SELECT` , `FILTER` , `OPTIONAL` , `ASK` , multi-triple `WHERE` blocks) and asks you to *write* queries against a small fixture knowledge graph. It is the retrieval move that converts recognition ("I've seen this query form") into recall ("I can write one"). Five queries, one Python file, autograded.

The drill runs entirely against an in-memory `rdflib` graph. **You do not need Docker or Fuseki running** — that is a Monday-lab concern. All you need is `rdflib` installed, which you already did in the reading's Setup section.

Setup

Submission flow — all Module 9 Week A assignments. Module 9 Week A uses a fork-and-submit flow (drill, lab, integration) instead of GitHub Classroom. GitHub is sunseting Classroom (full transition 2026-08-28). Steps 1–3 below are the fork-and-submit part; steps 4–5 are the same workflow you used in prior modules.

If you haven't already installed the M9 packages, go back to the [Setup: install before the concept session](#) section of the reading and complete the install before proceeding.

- Fork the template repo to your GitHub account.** Open github.com/LevelUp-Applied-AI/m9-d9a and click **Fork** (top right) → set the owner to your personal GitHub account → click **Create fork**. Leave "Copy the main branch only" checked.
- Enable Actions on your fork.** On your forked repo, click the **Actions** tab. GitHub disables workflows on forks by default — click **I understand my workflows, go ahead and enable them**. (One-time per fork. The autograder cannot run on your PR until this is done.)
- Clone *your* fork (not the upstream repo):**

```
git clone https://github.com/<your-github-username>/m9-d9a.git
cd m9-d9a
```
- Install dependencies (cached from the reading — should be fast):** `pip install -r requirements.txt`
- Create your working branch:** `git checkout -b drill-sparql`

The fixture

The repo includes `fixtures/mini_kg.ttl` — a tiny library catalog (~25 triples) of 5 books, with title, author, publication year, and optional topic. Inspect the file before writing queries. Two prefixes used throughout: `:` for the project namespace and `rdfs:` for `rdfs:label`.

A sketch of the shape (your file is the source of truth):

```
@prefix : <http://example.org/library/> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

:book1
  a :Book ;
  :title "The Pragmatic Programmer" ;
  :author :hunt , :thomas ;
  :year 1999 ;
  :topic "Software Engineering" .

:hunt rdfs:label "Andrew Hunt" .
:thomas rdfs:label "David Thomas" .

# ... four more books, two of which omit :topic
```

Your task

Complete `queries/drill.py`. Each function returns a SPARQL query as a string. The autograder will parse the fixture, execute each query, and check the result rows against the expected baseline.

```
# queries/drill.py - your task is to fill in the SPARQL string each function returns

def q1():
    """
    Q1 - Return all (book, title) pairs.

    Result: 5 rows. Variables in the SELECT: ?book ?title (in that order).
    """
    return "" # TODO

def q2():
    """
    Q2 - Return all books and their year, filtered to books published after 2010.

    Result: 1 row. Variables in the SELECT: ?book ?year.
    Use FILTER with > 2010 (strictly greater than, not >=).
    """
    return "" # TODO

def q3():
    """
    Q3 - Return all (book, author_name) pairs. author_name is the rdfs:label of
    the author resource.

    Result: 7 rows. Books with multiple authors produce one row per author.
    Variables in the SELECT: ?book ?author_name.
    """
    return "" # TODO

def q4():
    """
    Q4 - Return all books and their topic, with the topic OPTIONAL.

    Result: 5 rows (every book appears; ?topic is unbound for books with no
    :topic triple). Use OPTIONAL { ... }.
    Variables in the SELECT: ?book ?topic.
    """
    return "" # TODO

def q5():
    """
    Q5 - Return TRUE if any book has more than one :author triple; otherwise FALSE.

    Result: TRUE on this fixture. Use ASK. Match two distinct author bindings
    on the same book: FILTER (?a1 != ?a2).
    """
    return "" # TODO
```

How to run locally

```
pytest tests/ -v
```

The test file parses the fixture into an `rdflib.Graph`, calls each of your functions to get the SPARQL string, executes it, and asserts the result set.

If a query returns zero rows when you expect rows, re-read the Troubleshooting section of the reading. The two most common culprits are a missing `PREFIX` declaration and a typo in a predicate IRI.

Submission

- Commit and push your branch to *your fork*:

```
git add queries/drill.py
git commit -m "Drill 9A: SPARQL queries on fixture KG"
git push origin drill-sparql
```
- Open a Pull Request *within your fork*** — base branch `main`, compare branch `drill-sparql`. **Important:** the PR must stay inside your fork. When you click "Compare & pull request", GitHub defaults to "base repository: LevelUp-Applied-AI/m9-d9a" — change the **base repository** dropdown to your own fork (`<your-github-username>/m9-d9a`) before opening the PR. The autograder runs against your branch; check the CI status on the PR page to confirm green. If you see "Workflows are disabled" on the Actions tab, you missed Setup step 2 — enable them and push an empty commit (`git commit --allow-empty -m "ci: trigger"`) to re-run.
- Wait for the autograder workflow to run (~30 seconds).
- Paste your PR URL into TalentLMS → **Module 9** → **Drill 9A** to submit this assignment.

The autograder verdict (green check or red x) is what counts. You may push fixes and re-run the autograder as many times as you need before the deadline; resubmissions are accepted through Saturday of the week.

Tips

- Use the `PREFIX :` declaration that matches `fixtures/mini_kg.ttl`. Copy it from the top of the file rather than retyping.
- For Q3, you join through the author URI: one triple binds `?book :author ?a`, a second binds `?a rdfs:label ?author_name`. Both go in the same `WHERE` block.
- For Q4, the `OPTIONAL` block sits inside the `WHERE`. Put the mandatory `?book a :Book` triple first, then `OPTIONAL { ?book :topic ?topic }`.
- For Q5, `ASK` is the query form — not `SELECT`. The result is a boolean.

